

How Computers Work · Section 10

Virtual Machines and Containers

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. What Virtualization Is 1

2. Hypervisors and Virtual Machines 2

3. Containers, and How They Differ From VMs 2

4. Why Security Work Relies on Both 3

5. Snapshots and Clean States 4

6. The Limits of Isolation 4

Common Mistakes 5

Summary 6

Check Your Understanding 6

Practice 7

What You Will Learn

- What virtualization is.
- Hypervisors and virtual machines (VMs).
- Containers, and how they differ from VMs.
- Why security work relies on both.
- Snapshots, and resetting to a clean state.
- The limits of isolation.

Before You Start

Read lessons 5 and 6. You know the OS manages hardware, and that isolation (keeping programs apart) is a core security idea. This lesson is about running *whole separate systems* on one physical computer, safely isolated from each other. It is the technology behind the home lab you will build in course 08, and behind modern cloud computing. No code in this lesson; it is concepts, with the hands-on work saved for the lab and cloud courses.

---

1. What Virtualization Is

So far, one computer has run one operating system. **Virtualization** breaks that assumption: it lets one physical computer pretend to be several separate computers, each running its own operating system, all at the same time, and all kept apart from one another.

The trick is the same idea as virtual memory (lesson 6), but one level up. Virtual memory gave each *process* its own private view of memory. Virtualization gives each *operating system* its own private view of a whole *computer*: its own CPU, memory, disk, and network, all faked convincingly enough that the OS inside believes it is running on real, dedicated hardware. In reality, the one physical machine is shared among them.

Why is this so useful, especially in security?

- **Isolation.** Each virtual computer is sealed off from the others and, importantly, from the real host machine. Something that happens inside one, even something destructive, is contained.
- **Safety for experiments.** You can run dangerous or untrusted software (including real malware) inside a virtual computer, study it, and then throw the whole thing away, without risking your real machine.

- **Efficiency.** One powerful physical server can host many virtual servers, which is the foundation of cloud computing (course 34).

This single capability, running isolated, disposable, whole systems, is why virtualization underlies both your learning lab and a huge part of modern computing. The next topics explain the two main forms it takes.

2. Hypervisors and Virtual Machines

The most complete form of virtualization is the **virtual machine (VM)**: a full, simulated computer, running a complete operating system of its own, on top of your real one. On your single laptop, you could run a Windows VM and three different Linux VMs at once, each behaving like a separate PC.

Making this work is a special piece of software called a **hypervisor**: the manager that creates VMs and shares the real hardware among them, deciding which VM gets the CPU, how much memory each has, and so on. It is, in effect, a manager of operating systems, the way an OS is a manager of programs (lesson 5). Common tools you will use include **VirtualBox**, **VMware**, and, on Apple Silicon Macs, **UTM** (all covered in the Home Lab course).

The defining feature of a VM is that it is a **complete, independent computer**:

- It runs its **own full operating system** (its own kernel, lesson 5), separate from the host's.
- It has its **own virtual hardware**: virtual disk, virtual network card, virtual memory.
- It is **strongly isolated** from the host and other VMs by the hypervisor.

That strong isolation and completeness make VMs the standard safe place to handle dangerous software. When a malware analyst opens a real virus, they do it inside a VM: the malware infects the virtual machine, not the analyst's real computer, and afterward the whole VM is discarded. The cost of a VM is weight: because each runs a full operating system, VMs use a lot of memory and disk, and take time to start, like booting a whole computer, because that is exactly what they do.

3. Containers, and How They Differ From VMs

A **container** is a lighter form of isolation. Like a VM, it packages a program in its own isolated environment. Unlike a VM, a container does **not** run its own full operating system. Instead, all the containers on a machine **share the host's single kernel**, while each still gets its own isolated view of the file system, processes, and network.

The difference is best seen side by side:

	Virtual Machine	Container
Runs its own OS kernel?	**Yes**, a full OS each	**No**, shares the host's kernel
Weight	Heavy (GBs, slow to start)	Light (MBs, starts in seconds)
Isolation	Very strong (separate OS)	Strong, but weaker than a VM (shared kernel)
Best for	Running different OSes; strong isolation; malware	Packaging and shipping apps consistently

An analogy: a VM is like a separate house, with its own foundation, plumbing, and electricity (its own OS). A container is like a separate apartment in one building, it has its own locked door and rooms (isolated file system and processes), but it shares the building's foundation and utilities (the host's kernel). The apartment is far cheaper and quicker to set up, but it is less independent than a standalone house.

Containers (the best-known tool is **Docker**) transformed how software is built and deployed, because they make an app and everything it needs into one light, portable package that runs the same everywhere. That power, and its security implications, is a whole course of its own (Container and Kubernetes Security, course 35). The key fact to carry: **containers are lighter and faster than VMs but share the host kernel, which makes their isolation strong but not as strong as a VM's.**

4. Why Security Work Relies on Both

Virtualization is not a side topic in security; it is woven through the whole field. You will use VMs and containers constantly.

- **Your home lab is built on VMs** (course 08). You will run an attacker machine (often Kali Linux) and several vulnerable target machines as VMs on your own computer, on an isolated virtual network. This is what makes it *safe and legal* to practise attacks: everything is your own, contained, and disposable. The "only test what you own" rule (Security Fundamentals, lesson 9) is satisfied by design, because the whole lab is virtual machines you own.
- **Malware is analyzed in VMs** (course 29). You detonate real malicious software inside a disposable VM, watch what it does, and discard it, without endangering a real system.
- **The cloud is virtualization at scale** (course 34). Cloud servers are virtual machines and containers running on providers' hardware; securing them is a major field.
- **Defenders use containers and VMs** to run tools in clean, consistent, isolated environments.

So this lesson is unusually practical: the very next hands-on things you do in this program, building a lab, running tools, examining malware, all rest on virtualization. Understanding what a VM and a container *are*, and how they isolate, is the groundwork for all of it.

5. Snapshots and Clean States

One feature of virtual machines deserves its own mention, because it makes them so valuable for learning and for security: the **snapshot**. A snapshot saves the entire state of a VM, its disk and its memory, at a moment in time, so you can **return to that exact state later**, instantly.

Why this is powerful:

- **Experiment without fear.** Take a snapshot of a clean machine, then try something risky, break things, run malware, make a mess. When you are done, restore the snapshot, and the VM is instantly back to its clean state, as if nothing happened. This "undo for the whole computer" is perfect for learning.
- **Repeatable analysis.** A malware analyst infects a VM, studies the damage, then restores a clean snapshot to analyze the sample again from scratch, or to try a different approach, with an identical starting point every time.
- **Fast reset.** Instead of reinstalling an operating system (slow), you restore a snapshot (seconds).

This is why the Home Lab course will have you snapshot your machines before practising: you can attack and infect them freely, then reset to clean in seconds. The combination of **isolation** (nothing escapes the VM) and **snapshots** (reset to clean instantly) is what makes a virtual lab the ideal, safe playground for security skills.

6. The Limits of Isolation

Virtualization's isolation is strong, but, like every security control (Security Fundamentals, lesson 8), it is **not perfect**, and a good security person knows its limits.

The isolation between a VM (or container) and its host is enforced by software, the hypervisor or the kernel, and software can have flaws. An attack that breaks *out* of a virtual machine or container to reach the host, or another guest, is called a **VM escape** or **container escape**. These are rare and difficult (especially for VMs), and highly prized by attackers precisely because they defeat the isolation everyone relies on. They are a serious topic in cloud and container security.

The practical lessons:

- **Containers isolate less than VMs.** Because containers share the host kernel, a flaw in that kernel can affect all of them, and container escapes are generally more feasible than VM escapes. For handling truly dangerous or untrusted code, a **VM gives stronger isolation** than a container.
- **No isolation is absolute.** Even a VM is not a guarantee. When analyzing the most dangerous malware, professionals add extra layers: a dedicated physical machine, an isolated network, no sensitive data nearby, because defense in depth applies here too. Some malware even tries to *detect* that it is in a VM and behaves differently to evade analysis.
- **Keep the host and hypervisor updated.** The isolation is only as strong as the software enforcing it, so patching the hypervisor and host OS matters.

None of this undermines virtualization; it is still one of the most important security tools you have, and your lab depends on it. It simply means you treat isolation as *strong but not magical*: excellent for everyday safety and learning, and layered with extra caution when the stakes are highest.

You now understand how one computer can safely run many isolated systems, the foundation of your lab, malware analysis, and the cloud. You have come a long way: from bits, through the CPU and OS, to processes, files, programs, boot, and virtualization. The next lesson goes back down to the metal for a closer, hands-on look at memory with a little C, the ideas that the exploitation courses build on. Then the final lesson ties everything together on a real, running system.

Common Mistakes

1. Thinking a VM and a container are the same. A VM runs its own full OS (heavy, very strong isolation); a container shares the host's kernel (light, strong but weaker isolation).

2. Assuming isolation is perfect. VM and container escapes exist. Isolation is strong, not absolute, and must be layered for the most dangerous work.

3. Using a container for truly dangerous code. Containers share the host kernel, so a VM (or more) is the safer choice for real malware.

4. Forgetting snapshots. Snapshotting before risky work lets you reset to clean in seconds. It is the feature that makes a VM lab so forgiving.

5. Not updating the host/hypervisor. The isolation is enforced by that software; an unpatched flaw in it can be escaped. Keep it current.

Summary

Term	Remember
Virtualization	One physical computer runs several isolated systems at once
Virtual machine (VM)	A full simulated computer with its own OS; heavy; very strong isolation
Hypervisor	The software that creates VMs and shares the hardware (VirtualBox, VMware, UTM)
Container	Isolated like a VM but shares the host kernel; light and fast; strong but weaker isolation (Docker)
Why it matters	Your lab, malware analysis, and the cloud all run on virtualization
Snapshot	Saved VM state you can restore instantly; "undo" for a whole machine
Limits of isolation	VM/container escapes exist; isolation is strong, not absolute; layer it and patch it

Check Your Understanding

1. What does virtualization let one physical computer do, and how is it like virtual memory?
2. What is a virtual machine, and what is the hypervisor's job?
3. State the key difference between a VM and a container, and the main trade-off.
4. Why is it safe to analyze real malware inside a VM?
5. What is a snapshot, and why is it so useful for learning and for malware analysis?
6. What is a VM escape, and why is it prized by attackers?
7. For handling the most dangerous untrusted code, would you choose a container or a VM, and why?

Answers

1. It lets one physical computer run several separate systems, each with its own OS, at the same time and isolated from each other. Like virtual memory gives each process a private view of memory, virtualization gives each OS a private view of a whole computer.
2. A VM is a full, simulated computer running its own complete OS on top of the real one. The hypervisor is the software that creates VMs and shares the real hardware among them (deciding CPU, memory, etc.).
3. A VM runs its own full OS kernel; a container shares the host's kernel. Trade-off: VMs are heavier but more strongly isolated; containers are lighter and faster but less isolated.
4. Because the malware infects the isolated, disposable virtual machine, not the real host, and the whole VM can be discarded (or reset from a snapshot) afterward.

5. A saved copy of a VM's entire state (disk and memory) that you can restore instantly. It lets you experiment or infect a machine and then reset to a clean state in seconds, with an identical starting point every time.
6. An attack that breaks out of a VM to reach the host or other VMs. It's prized because it defeats the isolation everyone relies on, and it's rare and difficult.
7. A VM, because it runs its own separate kernel and gives stronger isolation than a container, which shares the host kernel (so a container escape is more feasible).

Practice

1. Using the house-vs-apartment analogy, explain to a friend the difference between a VM and a container, including why one is heavier.
2. You want to safely run a suspicious program you downloaded. Describe the steps you would take with a VM and a snapshot so your real computer stays safe.
3. Your future home lab will have an "attacker" machine and a "target" machine as VMs on one laptop. Explain why this setup lets you practise attacks legally and safely, linking it to the "only test what you own" rule.

A Worked Example

1. "A VM is like a separate house: it has its own foundation, plumbing, and electricity, that's its own operating system, so it's fully independent but heavy to build. A container is like an apartment in a building: it has its own locked door and rooms, isolated files and processes, but it shares the building's foundation and utilities, the host's kernel, so it's much lighter and quicker, but a bit less independent."
2. Create a VM with a fresh OS, take a clean snapshot, disconnect or isolate its network, then run the suspicious program inside the VM and observe it. When done, restore the snapshot to wipe any damage, and the program never touched my real computer.
3. Both machines are virtual machines running on my own laptop, on an isolated virtual network, so everything I attack is something I own and nothing reaches the real internet or anyone else's systems. That satisfies "only test what you own or have written permission to test" by design, because the entire lab is mine and contained, and snapshots let me reset it after each exercise.

Course mark icon from Material Symbols by Google, Apache License 2.0.